A Rectangular Logic Array

SHELDON B. AKERS, JR., SENIOR MEMBER, IEEE

Abstract—A rectangular logic array is described that can realize any combinational switching function. Straightforward analysis and synthesis procedures are described and the realizations of a number of special functions are given. These include threshold functions, parity functions, symmetric functions, and universal logic functions. Other properties of the array which are examined include diagnostic procedures, isolating defective cells, bounds on the array size, and possible implementations of the basic cell.

Index Terms—Cellular logic, combinational logic, logic arrays, symmetric functions, universal logic elements.

Introduction

THE rapid advances in microelectronics over the past few years have presented a variety of new challenges to the logic designer. Older design criteria have now been replaced by such concepts as regularity, ease of design, planarity, and testability. As a result a number of papers have appeared that explore the implications of these new requirements and propose various methods for achieving them [1]–[14]. This paper presents a particular two-dimensional array of logic cells that meets many of these requirements and possesses a number of interesting properties.

The array to be considered is composed of identical logic cells interconnected in a rectangular grid as shown in Fig. 1. Each cell has three inputs and two outputs. Two of the inputs to the cell are fixed connections—one to the cell immediately above and one to the cell immediately to the left. Likewise the two output leads are permanently connected to the two cells immediately to the right and immediately below. The third input to each cell is an *external input* on which an arbitrary input variable may be introduced. This will be assumed to be the only input or output under the designer's control. Thus the synthesis problem to be considered becomes that of introducing the proper input variables to the proper cells so that a given combinational switching function may be realized. (One way to visualize this structure is to picture the array as occupying a single two-dimensional "logic layer" with the external inputs being inserted to each cell from a second "input layer" directly above. See Fig. 2.)

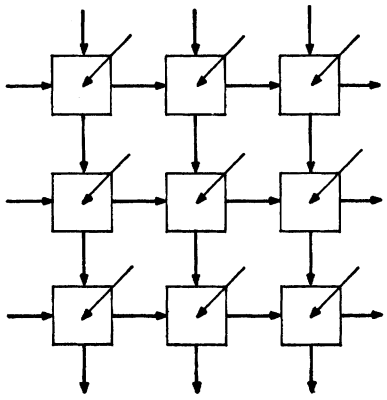


Fig. 1. The array.

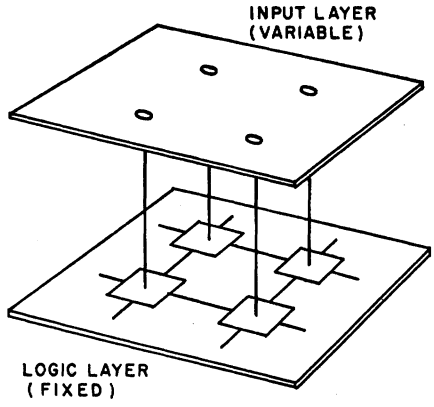


Fig. 2. Input and logic layers.

The Basic Cell

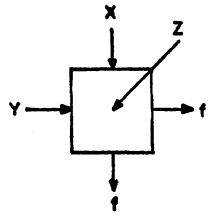
The selection of the precise logical function to be performed by the basic cell is, of course, a critical decision. Certainly it should be sufficiently complex to permit all logic functions to be realized, but at the same time it should be simple enough to be easily implemented and analyzed and should lead to reasonably straightforward design and diagnostic procedures.

A search through the various three-input switching functions is not particularly encouraging. The "powerful" functions lead to expensive implementations and complicated design rules. The "simple" functions either will not work at all or lead to excessively large arrays. However, there is a simple trick which, if it can be properly executed, can produce "several logic functions for the price of one." This is accomplished by defining the logical operation of the cells in such a way that when they are interconnected certain input combinations to the cells will never occur. This has the immediate effect of introducing DON'T CARE conditions which in turn permit any of several logic functions to describe the cell's operation.

To be specific, consider the cell shown in Fig. 3 and assume that it behaves as a three-input majority gate but never receives the input combinations of $X=1$, $Y=0$, $Z=0$ or $X=1$, $Y=0$, $Z=1$. (In other words, $X \leq Y$ always holds.) Then the resulting truth table

Manuscript received August 7, 1971; revised February 4, 1972. An earlier version of this paper was presented at the 12th Annual Symposium on Switching and Automata Theory, Michigan State University, East Lansing, Mich., October 1971.

The author is with the Electronics Laboratory, General Electric Company, Syracuse, N. Y. 13201.



X	Y	Z	f
0	0	0	0
1	0	0	-
0	1	0	0
1	1	0	1
0	0	1	0
1	0	1	-
0	1	1	1
1	1	1	1

Fig. 3. The cell.

will be as shown in Fig. 3 where the blanks denote DON'T CARE outputs.

Now, if it can be ensured that an input combination having $X=1$ and $Y=0$ will never occur (and this is yet to be shown), then there are *four* switching functions that precisely describe a cell's operation. These are obtained by simply filling in the blanks in the f column in Fig. 3 with 00, 01, 10, and 11, respectively. These functions are

$$f_0 = Y(X \vee Z)$$
$$f_1 = M(X, Y, Z) = XY \vee XZ \vee YZ$$
$$f_2 = X\bar{Z} \vee YZ$$
$$f_3 = X \vee YZ.$$

Before examining these functions further, let us see how it can be ensured that the $X=1$, $Y=0$ condition will never occur on the inputs to a cell, i.e., for any cell $X \leq Y$. Assuming that the cells are interconnected as shown in Fig. 1, all that needs to be done is to tie all the X inputs in the top row to 0 and all the Y inputs in the left column to 1. (Note that none of the external Z inputs are constrained.) Now it is easily seen that the $X \leq Y$ condition will be satisfied for all cells in the top row (since $X=0$) and for all cells in the left column (since $Y=1$). Moreover, it follows that for each of these cells an even stronger relationship holds. Namely, $X \leq f \leq Y$. We shall show that this stronger relationship holds for any cell. The argument proceeds inductively. Assume for a given cell that the condition has been shown to hold for the cell immediately above and the cell immediately to the left. (See Fig. 4.) Then, by hypothesis $X \leq f'$ and $f' \leq Y$. Thus, $X \leq Y$ and, from the truth table for the cell, $X \leq f \leq Y$. Thus, this cell and hence all of the cells must satisfy the $X \leq f \leq Y$ condition.

With all of the X and Y inputs permanently fixed, i.e., tied to neighboring cells or to 0 or to 1, it will often be convenient to omit these inputs from future diagrams and simply show the matrix of the values of the external Z inputs. (See Fig. 5.) Unless otherwise indicated, it will be assumed that the function being realized is given by the output of the cell in the lower right corner of the array and that all signal flows are left to right and top to bottom.

Returning to the four functions that can describe the cell operation, f_0 and f_3 both suggest simple ways of

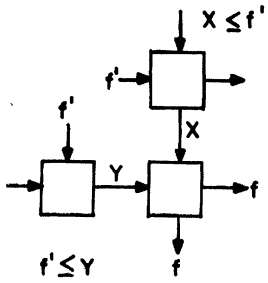


Fig. 4. $X \leq f \leq Y$ relationship.

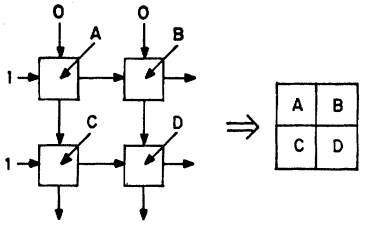


Fig. 5. The input matrix.

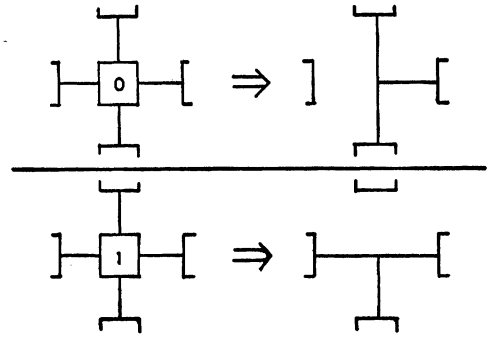


Fig. 6. "Shorting" a cell.

implementing the cell and these implementation possibilities will be examined in a later section. Neither of these functions appears to yield particularly attractive design procedures especially when compared with f_1, the three-input majority function. (The reasons for this will become apparent shortly.) The function $f_2 = X\bar{Z} \vee YZ$, however, is definitely of interest. For one thing, it is precisely the Shannon function that is used to expand a Boolean function about a given input variable, (e.g., $f(X_1, X_2, \cdots, X_n) = \bar{X}_1 f(0, X_2, \cdots, X_n) \vee X_1 f(1, X_2, \cdots, X_n)$). Also, if we set $Z=0$, then f_2 reduces to X, while with $Z=1$, f_2 becomes Y. Thus by tying Z to 0 or to 1, a cell can be "shorted" either vertically or horizontally. This property (shown schematically in Fig. 6) will prove quite useful in following the signal flow through an array, and also for detecting and isolating defective cells.

Calculating the Array Output

The first step in developing a systematic design procedure is to find a simple procedure for calculating the binary output of an array, given the binary inputs. Assume that a 4×5 array has the set of binary input values shown by the input matrix in Fig. 7. What will be the output of the array, i.e., the output of the cell in the lower right corner?

1	0	0	1	0
0	0	1	1	1
1	1	0	1	0
0	1	1	0	1

Fig. 7. A set of binary inputs.

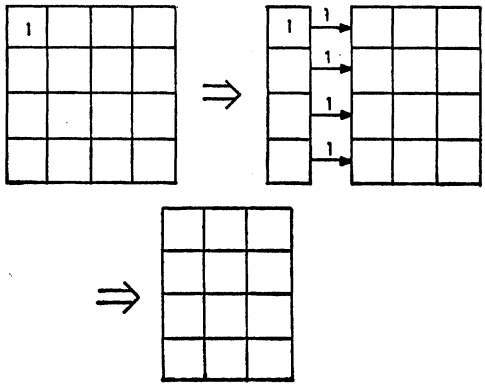


Fig. 8. Result of 1 on upper left cell.

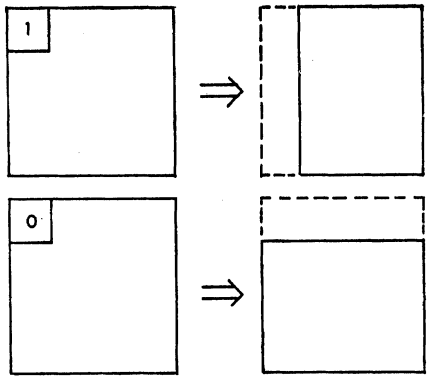


Fig. 9. "Stripping" process.

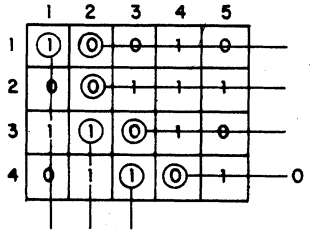


Fig. 10. Finding the array output.

Since each cell behaves like a three-input majority gate, one obvious way to calculate the array output would be to start with the cell in the upper left corner (all of whose inputs are known) and successively calculate each of the individual cell outputs until the final output cell is reached. However, a much quicker method is available. Assume (as in this example) that the upper left cell has an external input of 1. Then since it also receives a 1 from the left, its output will be 1. Now consider the cell immediately below it. This cell receives a 1 from the cell above and also a 1 from the left. Thus its output will also be 1, *regardless of its external input.* A similar argument holds for each lower cell in the left-hand column. Thus, an external input of 1 into the upper left cell forces *all* cells in the left-hand column to have outputs of 1. But this is precisely the case that would result if the left-hand column were deleted from the original matrix and the smaller matrix considered. (See Fig. 8.)

A similar argument shows that when the external input to the upper left cell is 0, the top *row* of the input matrix may be deleted, and the output of the resulting smaller input matrix will be the same. Fig. 9 summarizes this "stripping" procedure. Thus, to calculate the binary output of an array from the binary input matrix, rows and columns are removed from the matrix according to the above rules until the matrix is completely deleted. The bit that completes the process, i.e., causes the removal of the row or column containing the lower right output cell—gives the binary value of the array output. (Note that this process actually determines the output of every cell since the value of a cell's output will be identical with the bit that caused its removal.)

Consider again the example of Fig. 7 and assume Z_{ij} is the input to the cell in the ith row and jth column. $Z_{11}=1$ causes column 1 to be deleted, $Z_{12}=0$ causes the rest of row 1 to be deleted, $Z_{22}=0$ deletes the rest of row 2, $Z_{32}=1$ deletes the rest of column 2, $Z_{33}=0$ deletes the rest of row 3, $Z_{43}=1$ deletes the rest of column 3, and finally $Z_{44}=0$ deletes the rest of row 4 including the output cell whose output must therefore be 0. Fig. 10 shows this process.

1- AND 0-PATHS

A close examination of the foregoing procedure reveals that whenever the output of an $m \times n$ array is 0 there must exist a 0 in each of the m rows of the input matrix and moreover the 0 in row i must be at least as far to the right as the 0 in row i-1. Such a set of 0's will be called a *0-path* and defined formally as follows.

Given an $m \times n$ binary input matrix Z_{ij} a set of m 0-inputs,

$$Z_{1j_1}, Z_{2j_2}, \cdots, Z_{mj_m}$$

will be called a *0-path* if and only if

$$j_1 \leq j_2 \leq \cdots, \leq j_m.$$

Likewise, a *1-path* in an input matrix will be a set of n 1-inputs,

$$Z_{i_1 1}, Z_{i_2 2}, \cdots, Z_{i_n n}$$

having $i_1 \leq i_2 \leq \cdots, \leq i_n$.

(That is, a 1 in each column and each 1 at least as low as the 1 in the column to the left.) Fig. 11 shows some typical 0- and 1-paths. Also, see the 0-path in Fig. 10. Note that in any binary matrix one and only one of these two types of paths will exist.

Now the problem of synthesizing a switching function $f(A, B, C, \cdots)$ can be stated as follows. Find an input matrix of the literals, i.e., A, $\overline{A}$, B, $\overline{B}$, $\cdots$ such that

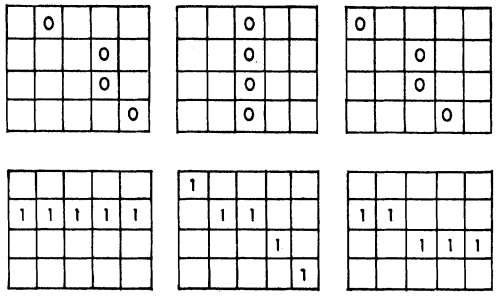


Fig. 11. Some 0- and 1-paths.

every input combination for which $f=1$ induces a 1-path in the matrix and every input combination for which $f=0$ induces a 0-path.

It also follows that given an input matrix of the literals, the function that it realizes can be found by simply examining the possible 1-paths (or 0-paths). Consider the 2×2 input matrix of Fig. 5. There are three and only three possible 1-paths: when A and B are 1; when A and D are 1; or when C and D are 1. In other words

$$f = AB \vee AD \vee CD.$$

Likewise, the three possible 0-paths yield a product of sums form of the same function:

$$f = (A \vee C)(A \vee D)(B \vee D).$$

It can be shown inductively that in general an $m\times n$ input matrix will have

$$\binom{m+n-1}{n} \text{ 1-paths}$$

and

$$\binom{m+n-1}{m} \text{ 0-paths.}$$

Of course many of the 1-paths may yield identical product terms (if the same set of literals appear) or no term at all (if (say) A and $\overline{A}$ both appear in the path), and likewise for the 0-paths.

Before proceeding to some actual synthesis procedures, several interesting properties of these input matrices may be noted. If an input matrix of literals is rotated 180°, it is easily seen that all 1- and 0-paths are preserved and hence that the same function will be realized. However, if the *transpose* is generated, all 1-paths become 0-paths and vice versa so that the *dual* of the function results. Likewise, by DeMorgan's rule taking the transpose and then complementing all of the literals results in the *complement* of the original function. (Fig. 12 summarizes these properties.)

A Synthesis Procedure

The problem of ensuring that an input matrix will produce the required 1- and 0-paths for a given function may at first glance seem rather forbidding, but there are several straightforward ways of accomplishing this. Assume the function

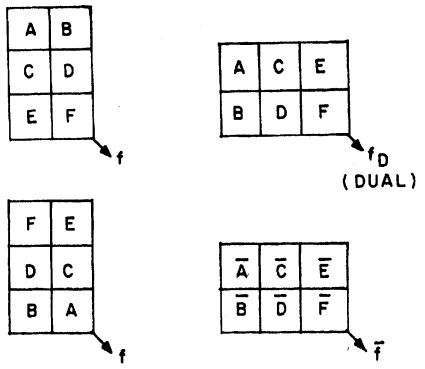


Fig. 12. Forming the dual and complement.

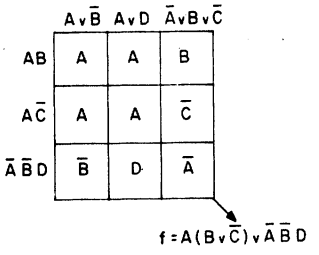


Fig. 13. Synthesizing a function.

$$f = A(B \vee \overline{C}) \vee \overline{A}\,\overline{B}D$$

is to be synthesized. One method is to first expand this function into its sum of products and product of sums forms:

$$f = AB \vee A\overline{C} \vee \overline{A}\,\overline{B}D$$
$$f = (A \vee \overline{B})(A \vee D)(\overline{A} \vee B \vee \overline{C}).$$

Next an input matrix is constructed having one row for each product and one column for each sum. Each row (column) is then labeled with a different product (sum). Now since any product must have a literal in common with any sum, it follows that for every square in the matrix there exists at least one literal that appears in both the product and sum associated with that square. Enter any such literal into the square. (See Fig. 13.) The resulting input matrix realizes the function.

To see this, consider any binary input combination for which the function is to be 1. This combination must induce 1's on one of the products and hence on the entire row to which this product is assigned. Thus, the binary input matrix will have a row of all 1's which is a 1-path and hence the output will be 1. A similar argument shows that whenever f should be 0 a column of 0's will appear. Fig. 14 shows some two- and three-variable functions realized by this procedure.

If the function to be synthesized is specified in truth table form, essentially the same procedure may be followed except that now the rows are labeled with binary input combinations (α's) for which $f=1$ and the columns with combinations (β's) for which $f=0$. (DON'T CARE input combinations are simply omitted.) Z_{ij} is now set equal to one of the literals which is 1 in α_i and 0 in β_j. (There will always be at least one.) Fig. 15 illustrates this procedure. Actually this is precisely the

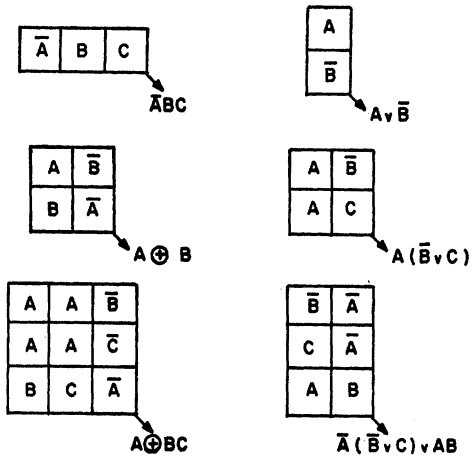


Fig. 14. Some two- and three-variable functions.

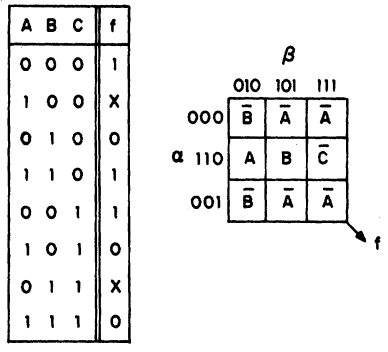


Fig. 15. Synthesis of a truth table.

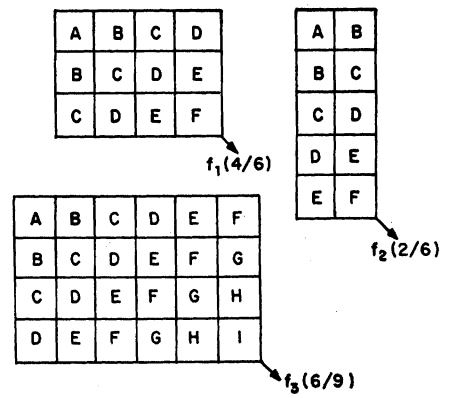


Fig. 16. Some "regular" arrays.

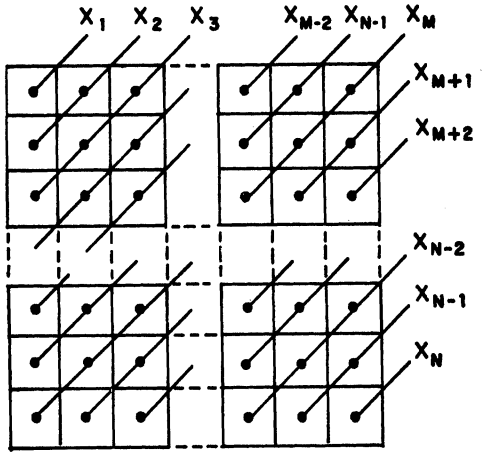


Fig. 17. The "M out of N" array.

technique which was used to generate "logic diagrams" [5]. Accordingly, all of the rules given there for initially reducing the number of rows in the truth table (and hence the size of the resulting array) may be applied. This method is especially useful when there are a number of DON'T CARES in the truth table.

Some Special Arrays

While the above procedure has the virtue of being quite easy to apply (and as such provides a simple proof that any function can be realized with the array), it has at least two rather serious drawbacks for larger numbers of variables. First, it makes very inefficient use of the input matrix since it only generates 1- and 0-paths which are complete rows or columns. (With a 4×5 matrix, for example, only 4 of the 56 possible 1-paths would be explicitly used and only 5 of the 70 possible 0-paths.) The result, of course, is that the resulting arrays are often far larger than necessary. A second criticism is that the resulting input matrix often has the literals "sprinkled about" in a nearly random fashion, thus making the problem of routing the various input variables to the correct cell quite difficult.

For these reasons, we shall leave the synthesis problem for the moment and look instead at some "more regular" arrays and try to determine what functions they generate. In this way we may gain some insight into how a more efficient synthesis procedure might be developed.

Consider the input matrices in Fig. 16. Since each diagonal contains only a single literal, the routing problem for the input signals is certainly simplified. But are the functions that these matrices generate of any particular interest?

If the possible 1-paths for f_1 are traced out, it is seen that there are exactly 15 such paths and in fact one for every set consisting of four of the six literals. Likewise, there are 20 0-paths, one for every possible set of three of the six literals. Thus, f_1 is the "four out of six" function, i.e., the six-variable function whose output is 1 if and only if at least four of its six input variables are 1. Similar analyses show that f_2 is the "two out of six" function and f_3 is the "six out of nine" function. (The 4×6 array for f_3 dramatically illustrates the possible inefficiency of the previous product versus sum method. That approach for f_3 would yield an array with just over 10 000 cells!)

The generalization of this procedure to an arbitrary "M out of N" function should be obvious. The resulting array will have $N-M+1$ rows and M columns. (See Fig. 17.) Previous authors have described essentially this same configuration using 3-input majority gates [3], [14]. By "overlapping" these arrays, a multiple-output "ones counter" can also be obtained (Fig. 18) where $f_i = 1$ if and only if at least i of the n inputs are 1.

Clearly any threshold function having positive integer weights can also be realized with this type of array.

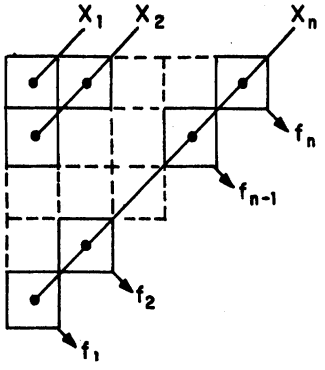


Fig. 18. A "ones counter."

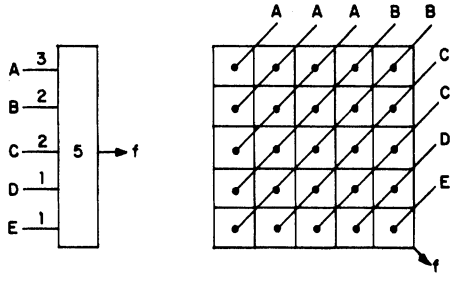


Fig. 19. Realizing a threshold function.

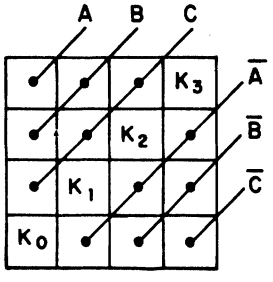


Fig. 20. Any symmetric function of three variables.

Variables are simply introduced as many times as their weight requires. Fig. 19 shows the (3, 2, 2, 1, 1) threshold function having a threshold of 5.

Another useful array configuration is that shown (for three variables) in Fig. 20. Here the uncomplemented literals are introduced on the upper diagonals and their complements on the lower diagonals. The inputs along the main diagonal are tied to binary constants, K_0, K_1, K_2, and K_3. Now it is easily shown that whenever the arithmetic sum of $A+B+C$ equals i the output of the array will be K_i. Thus, this array will realize any *symmetric* function of A, B, and C. (Recall that a function is *symmetric* in A, B, and C if and only if their arithmetic sum completely determines the output. Hence, K_i is simply set to the value of the output required when $A+B+C=i$.) The generalization to any number of variables should be obvious.

A modest improvement on the size of this array can be effected as shown in Fig. 21. Here C_i denotes the value of the function when $A+B=i$. Clearly this value must be C, $\overline{C}$, 0, or 1, any of which is a permissible input. Again the generalization to n variables (an $n\times n$ array) should be obvious. Note that the function need be symmetric in only $n-1$ of the variables. Fig. 22 shows the symmetric four-variable parity function realized by these two methods.

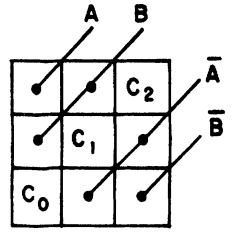


Fig. 21. Smaller version of three-variable symmetric function.

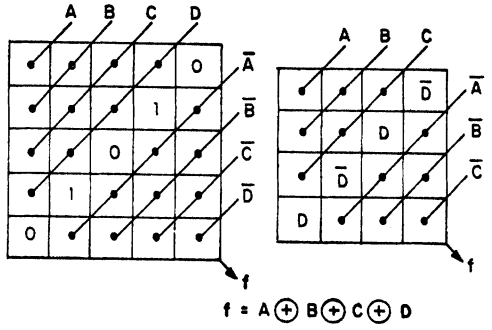


Fig. 22. Two realizations of the four-variable parity function.

Universal Logic Arrays

Fig. 23 shows two universal logic arrays for two and three variables. ($g_0=g(A, 0)$, $g_1=g(A, 1)$, $f_0=f(A, 0, 0)$, $f_1=f(A, 0, 1)$, etc.) Fig. 24 shows how a universal four-variable array can be constructed. Since any function $f(A, B, C, D)$ may be written as $Dh_1 \vee \overline{D}h_0$ where $h_1=f(A, B, C, 1)$ and $h_0=f(A, B, C, 0)$, the 2 three-variable arrays for h_1 and h_0 can first be constructed. These two arrays are then "overlapped" as shown with D and $\overline{D}$ being inserted on the new diagonals at the upper left and lower right as indicated. It is easily shown that this 5×5 array realizes any four-variable function.

In a similar manner, a 9×9 universal five-variable array can be constructed by overlapping two of these 5×5's. In general an n-variable universal array will be of dimensions $(2^{n-2}+1)\times(2^{n-2}+1)$ or a total of $2^{2n-4}+2^{n-1}+1$ cells. This gives an upper bound on the number of cells for an arbitrary function of n variables.

To obtain a lower bound, we observe that if an array of C cells will realize any function of n variables, then there are $2n+2$ choices for the external input to each cell. (n uncomplemented variables, n complemented variables, plus 0 and 1.) Thus, if 2^{2^n} different functions can be realized, then

$$(2n+2)^C \geq 2^{2^n}$$

or

$$C \geq 2^n/\log_2(2n+2).$$

These bounds can hardly be called "tight" and for a very good reason. The lower bound shows that the *area*

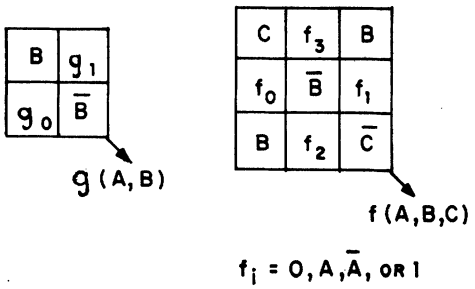


Fig. 23. Universal arrays for two and three variables.

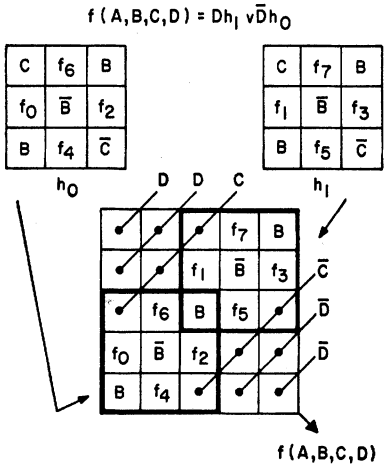


Fig. 24. Construction of a four-variable universal array.

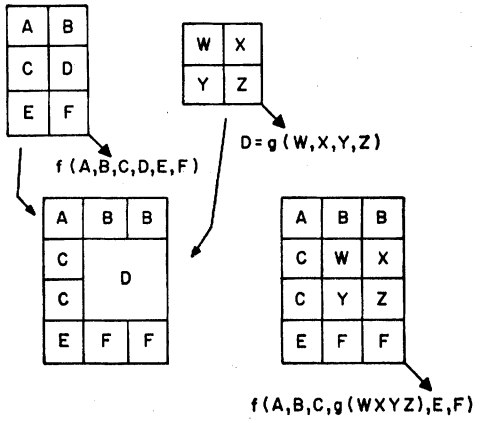


Fig. 25. Embedding an array.

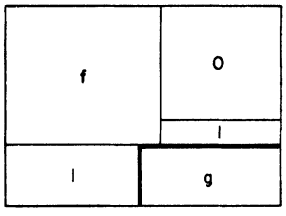


Fig. 26. ORing two arrays.

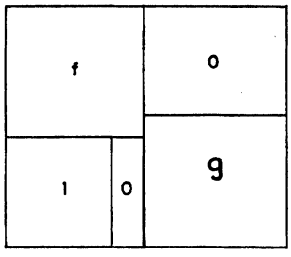


Fig. 27. ANDing two arrays.

of the array must almost *double* each time a new variable is added. However, the described procedure at each step almost doubles the *sides*. Certainly the lower bound can be raised considerably by taking into account the various symmetries of the array, but the upper bound seems a more likely candidate for a dramatic reduction.

Embedding Arrays

Another interesting feature of these arrays is the fact that one array can be easily *embedded* within another. This is especially useful when a given function can be decomposed into smaller functions. Assume, for example, that a function $f(A, B, C, D, E, F)$ has been realized as shown in the upper left of Fig. 25, and that D is itself the function $g(W, X, Y, Z)$ realized by the array in the upper right. To embed g within f, it is merely necessary to replicate the rows and columns containing D in the f array so that the array of D cells assumes the exact dimensions of g. Now g may be inserted directly into f. (See Fig. 25.) Again, an examination of the 0- and 1-paths shows that this procedure is indeed feasible.

Interconnecting Arrays

When a logic designer is faced with a switching function of (say) 20 variables to be synthesized, he seldom, if ever, attacks the function in its entirety. Instead he "builds up" the function by synthesizing smaller "blocks" that are then gated together with other blocks to realize the complete function. A similar approach is possible with these arrays. Assume the arrays for two separate functions f and g have been found and that the function $h = f \vee g$ is desired. Fig. 26 shows how this may be accomplished. (A constant inside a rectangle will indicate that all of the external inputs to the cells within that area are to be tied to that constant.) To see that this array does generate $f \vee g$, assume $f = 1$. This will force all of the cells in the row immediately above the g array (Fig. 26) to have outputs of 1 which together with the 1's from the left of the g array will force every cell in the g array to have a 1 output. When $f = 0$, 0's will be introduced to the top of the g array (1's remaining on the left) so that its output will be g itself as required. A similar argument holds for the $f \cdot g$ array of Fig. 27.

In both of these realizations (Figs. 26 and 27), the only purpose of the "0-cells" in the upper right was to propagate 0's to the top edge of the lower array. Hence, if convenient, these cells may be removed providing the resulting top edge of the lower array has all its cells tied to 0. Likewise, for the "1-cells" in the lower left. Fig. 28 shows the realizations of the familiar sum of products

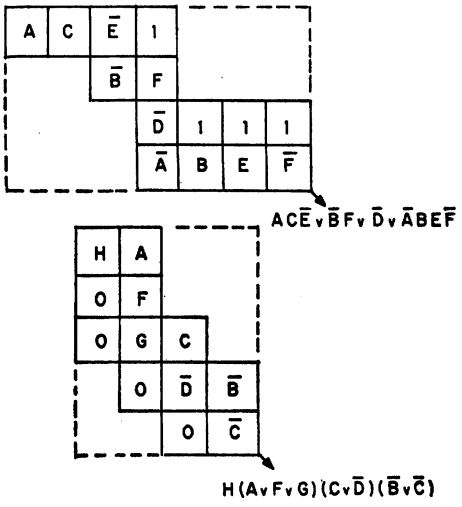


Fig. 28. Sum of products and product of sums realizations.

and product of sums forms with the "surplus" cells removed.

Testing an Array

An important requirement on any large cellular array is that it be easily checked for defective cells. A number of authors have investigated this problem [10]–[13]. Because this array can be implemented with any of four different types of cells, different diagnostic procedures will be necessary depending on the particular cell type employed. This is especially true because a cell failure may cause the important "$X \leq Y$ condition" to be violated on other cells and hence it will be necessary to know how they will behave in this case (i.e., when $X=1$ and $Y=0$). An examination of all of these possibilities is beyond the scope of this paper.

There are, however, two types of cell failures that will not cause the "$X \leq Y$ condition" to be violated, namely, the *external* Z input of a cell being stuck at 0 or 1. It will be instructive, therefore, to look at how these two failure types might be detected and isolated since similar techniques will apply to the more complicated case of *internal* failures.

Fig. 29 shows how the "shorting" scheme of Fig. 6 may be employed to generate "sensitized paths" to and from a given cell. Now when the X input is tied to 0 and the Y input to 1, a "stuck-at-0" or "stuck-at-1" failure on the Z input can be readily detected.

If an external input failure has been identified, the isolation of that cell from the rest of the array can be accomplished by applying the previous "shorting" scheme to the two cells which it drives. (See Fig. 30.) This essentially "cuts" the two outputs of the cell so that it is isolated from the rest of the array. The immediate consequence of this is, of course, to complicate the synthesis process. However, the other cells will behave exactly as previously described, the only difference being that three of the inputs in the input matrix are permanently fixed.

A more expensive way to isolate such a cell, but one

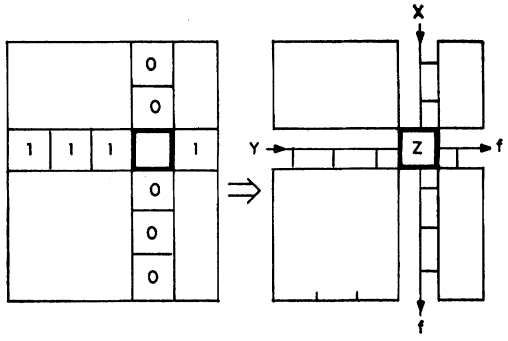


Fig. 29. Testing a cell.

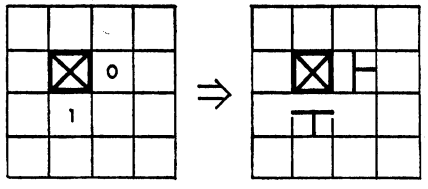


Fig. 30. Isolating a defective cell.

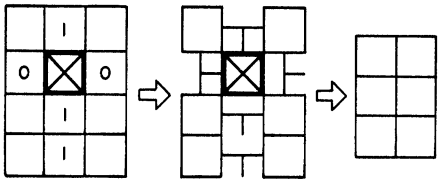


Fig. 31. "Removing" a row and column.

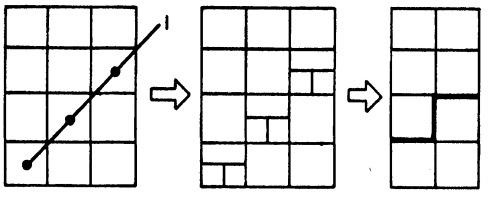


Fig. 32. "Removing" a diagonal.

that preserves all of the previous synthesis procedures, is to "short" all cells in the same row or column as the defective cell. (See Fig. 31.) For design purposes, this results in a smaller (by one row and column) array on which all of the previous methods still apply. Note that this method will work regardless of the value at which the external input is stuck.

Fig. 32 shows still another method of "removing" a cell with a defective external input. Here all of the cells in its diagonal are tied to the value (say 1) at which the defective input is stuck. Now it is easily seen that this 4×3 array will behave precisely like the 4×2 array shown at the right. Likewise, with a "stuck-at-0" failure, tying all external inputs on the same diagonal to 0 will result effectively in a 3×3 matrix. In summary, a single "Z-stuck-at-1" failure can be "removed" at the expense of one column of the matrix, while a "Z-stuck-at-0" failure will cause the loss of one row. Thus, a large "almost perfect" array can often be "saved" with only a slight reduction in dimensionality.

Implementing the Basic Cell

From a logical standpoint, the only requirement on an implementation of the basic cell is that it give the outputs shown in Fig. 3 for the six "care" input combinations. However, from an electrical point of view, a variety of other considerations such as timing, signal attenuation, etc., are involved. These, of course, will vary with the particular technology involved.

We shall be content, therefore, to merely give some simple "logical" implementations of the cell and leave the (more difficult) electrical problems to the hardware specialists. As noted previously, two simple ways of describing the cell's operation are given by

$$f_0 = Y(X \vee Z)$$

and

$$f_3 = X \vee YZ.$$

Thus, there are 2 two-gate AND–OR realizations possible (Fig. 33).

Of perhaps more practical interest are the NAND and NOR realizations. These require three gates (strictly speaking, two gates and an inverter) as shown in Fig. 34. Of course, if a three-input two-output majority gate is available, this alone will realize the cell.

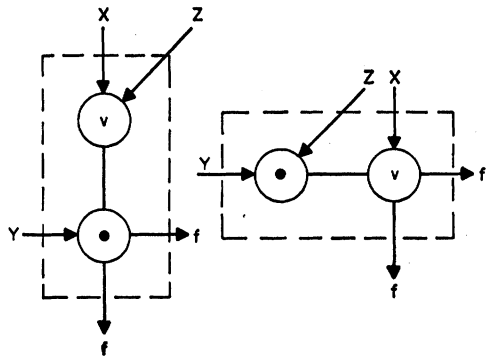


Fig. 33. AND/OR realizations of basic cell.

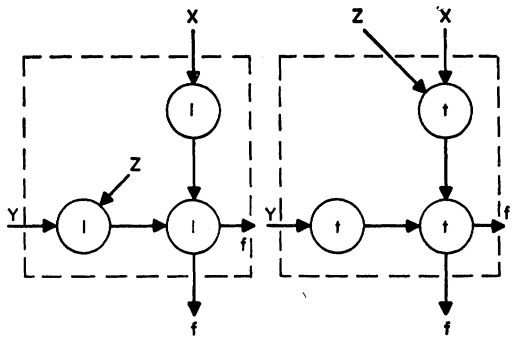


Fig. 34. NAND and NOR realizations.

Multiple-Output Functions

The problem of synthesizing several different functions on a single array is made difficult by the fact that the outputs from many of the cells in a given array bear precise logical relationships to each other. In particular, it follows from the "$X \leq f \leq Y$" relationship that the output of cell i, j "covers" the output of every cell i', j' where $i' \leq i$ and $j' \geq j$ (i.e., each cell to the upper right). Likewise, its output is "covered" by the output of each of the cells to the lower left. Thus, if a multiple-output function is to be realized, the outputs can in general be taken only from cells that are not logically related.

A possible approach in this regard is to note that a cell's output *can* be made independent of those to its upper left and lower right. Hence, one alternative is to assume that each cell has *three* outputs—the third possibly being accessible from below.

Several familiar multiple-output functions now become straightforward. The "carry function," for example, which is commonly used with n-bit parallel adders requires that C_i $(i=0, 1, \cdots, n-1)$ be the carry bit from the ith stage. Since it may be defined recursively as

$$C_i = M(A_{i-1}, B_{i-1}, C_{i-1})$$

where $C_0=0$ and A_i and B_i denote the corresponding bits of the numbers being added, it follows that an array such as that in Fig. 35 will generate all the required C_i (from the cells on the main diagonal).

Many of the previously described synthesis procedures would need considerable extension to accommodate multiple outputs. However, at least one—the product versus sum approach—is easily modified. Given two functions, f_1 and f_2 in their product of sums and sum of products forms, let m_i denote the set of products appearing only in f_1, m_{12} the set in both, and m_2 the set only in f_2 (likewise for the sums, M_1, M_{12}, and M_2). Now if the products and sums are assigned to rows and columns as shown in Fig. 36 and the corresponding literals

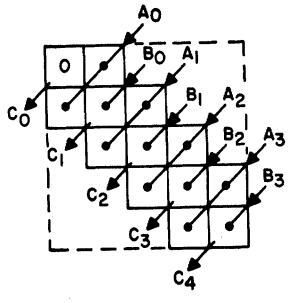


Fig. 35. Carry generation.

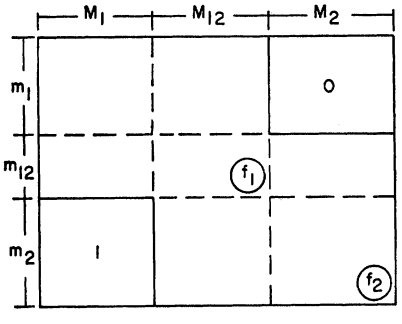


Fig. 36. A multiple-output procedure.

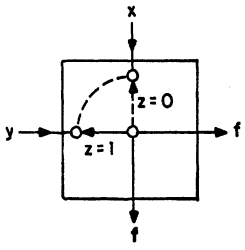


Fig. 37. Basic cell as a double-throw switch.

inserted as before, it is easily seen that the outputs of the two circled cells will generate the required functions.

Final Remarks

One of the most interesting features of the array that has been described is the fact that whenever a function has some "simplifying" property such as being symmetric, unate, decomposable, etc., a corresponding simpler array directly reflecting this property can be found. Thus, while a 17×17 array may only be capable of realizing *all* functions of up to six variables, it can also accommodate any symmetric function of up to 17 variables, or any majority function of up to 33 variables.

Little attempt has been made to find the minimum number of cells necessary to realize a given function since this emphasis on minimization is today seldom of primary importance. (Many, in fact, doubt that it ever was.) Rather the emphasis has been on "regular" arrays —in particular, ones where most of the (lower left to upper right) diagonals have all of their cells tied to a common input variable. (Note that this not only simplifies the routing of the input variables, but also the timing problems since all cells on the same diagonal are on the same "timing level" relative to the output cell.)

Finally, the "$X\bar{Z} \vee YZ$" version of the cell should be mentioned. Not only does this interpretation (Fig. 6) yield an efficient method for analyzing the various signal flows within the matrix, it also suggests that all of the foregoing synthesis and analysis procedures could be developed on a purely topological basis. Also, if the reader should wish to build his own array, he can implement the basic cell with one single-pole double-throw switch (Fig. 37).

References

[1] F. C. Hennie, *Iterative Arrays of Logical Circuits*. Cambridge, Mass.: M.I.T. Press; New York: Wiley, 1961.
[2] W. H. Kautz, "The realization of symmetric switching functions with linear-input logical elements," *IRE Trans. Electron. Comput.*, vol. EC-10, pp. 371–378, Sept. 1961.
[3] S. Amarel, G. Cooke, and R. O. Winder, "Majority gate networks," *IEEE Trans. Electron. Comput.*, vol. EC-13, pp. 4–13, Feb. 1964.
[4] R. C. Minnick, "Cutpoint cellular logic," *IEEE Trans. Electron. Comput.*, vol. EC-13, pp. 685–698, Dec. 1964.
[5] S. B. Akers, Jr., "A diagrammatic approach to multilevel logic synthesis," *IEEE Trans. Electron. Comput.*, vol. EC-14, pp. 174–181, Apr. 1965.
[6] H. S. Stone and A. J. Korenjak, "Canonical form and synthesis of cellular cascades," *IEEE Trans. Electron. Comput.*, vol. EC-14, pp. 852–862, Dec. 1965.
[7] R. C. Minnick, "Survey of microcellular research," *J. Ass. Comput. Mach.*, vol. 14, pp. 203–241, Apr. 1967.
[8] M. Yoeli and J. Turner, "Decomposition of group functions with applications to two-rail cascades," *Inform. Contr.*, vol. 10, pp. 565–571, June 1967.
[9] B. Elspas and H. S. Stone, "Decomposition of group functions and the synthesis of multirail cascades," in *Conf. Rec. 8th Annu. Symp. Switching and Automata Theory* (Austin, Tex.), Oct. 1967, pp. 184–196.
[10] W. H. Kautz, "Testing for faults in cellular logic arrays," in *Conf. Rec. 8th Annu. Symp. Switching and Automata Theory* (Austin, Tex.), Oct. 1967, pp. 161–174.
[11] A. Mukhopadhyay, "Unate cellular logic," *IEEE Trans. Comput.*, vol. C-18, pp. 114–121, Feb. 1969.
[12] S. S. Yau and M. Orsic, "Fault diagnosis and repair of cutpoint cellular arrays," *IEEE Trans. Comput.* (Short Notes), vol. C-19, pp. 259–262, Mar. 1970.
[13] P. R. Menon and A. D. Friedman, "Fault detection in iterative logic arrays," *IEEE Trans. Comput.*, vol. C-20, pp. 524–535, May 1971.
[14] R. H. Canaday, "Two dimensional iterative logic," M.I.T. Electron. Syst. Lab Rep. ESL-R-210, 1964.